



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico Grep

31 de Octubre de 2025

Lenguajes Formales, Autómatas y Computabilidad

Integrante	LU	Correo electrónico
Mateo Ziffer	282/24	mateoziffer@gmail.com
Nazareno Olazarri	1017/23	Nazareno_Olaz@hotmail.com
Martín Kraisman	1305/23	martin.kraisman@gmail.com
Completar Nico	LU	Mail



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - Pabellón I

Intendente Güiraldes 2160 - C1428EGA

Ciudad Autónoma de Buenos Aires - Argentina

Tel/Fax: (54 11) 4576-3359

<http://exactas.uba.ar>

1. Decisiones de diseño

Al inicializar la clase `RegEx` generamos su AFD mínimo respectivo y lo guardamos en un nuevo atributo llamado `_afd` para luego al hacer cada `match` poder usarlo y no tener que generarlo cada vez.

Para varios de los algoritmos que implementamos decidimos usar los sets de Python. Aunque estos sets tienen limitaciones, específicamente que no se pueden crear sets de sets, y que no se puede iterar un set y agregar o sacar valores en el medio de la iteración, por esto utilizamos cuando es necesario los frozensets, conjuntos cuyos elementos no son modificables.

- no modificamos automatas en determinar y minimizar, sino que creamos nuevos y los retornamos

2. Eleccion de algoritmos

Al generar el AFND a partir de la expresión regular utilizamos el algoritmo de Thompson basandonos en la inducción del teorema visto en clase. Se puede decir que en la implementación extendemos el invariante con tener los estados del autómata normalizados en cada paso.

Para hacer la minimización, vimos en clase los algoritmos de Moore, Brzozowski y de Hopcroft. De estos usamos el de Hopcroft porque tiene la mejor complejidad temporal, $O(ns \log n)$ y porque se basa en hacer operaciones entre conjuntos, las cuales ya tenemos implementadas con los sets de Python. Este algoritmo requiere que el autómata no tenga estados accesibles, por lo tanto agregamos un método `minimize` a la clase AFD que primero crea un autómata equivalente sin estados inaccesibles y luego aplica al algoritmo de Hopcroft sobre el mismo.

3. Benchmarks y eficiencia

4. Cambios posibles

- sort estados con counting sort - agregar estados n^2 a n